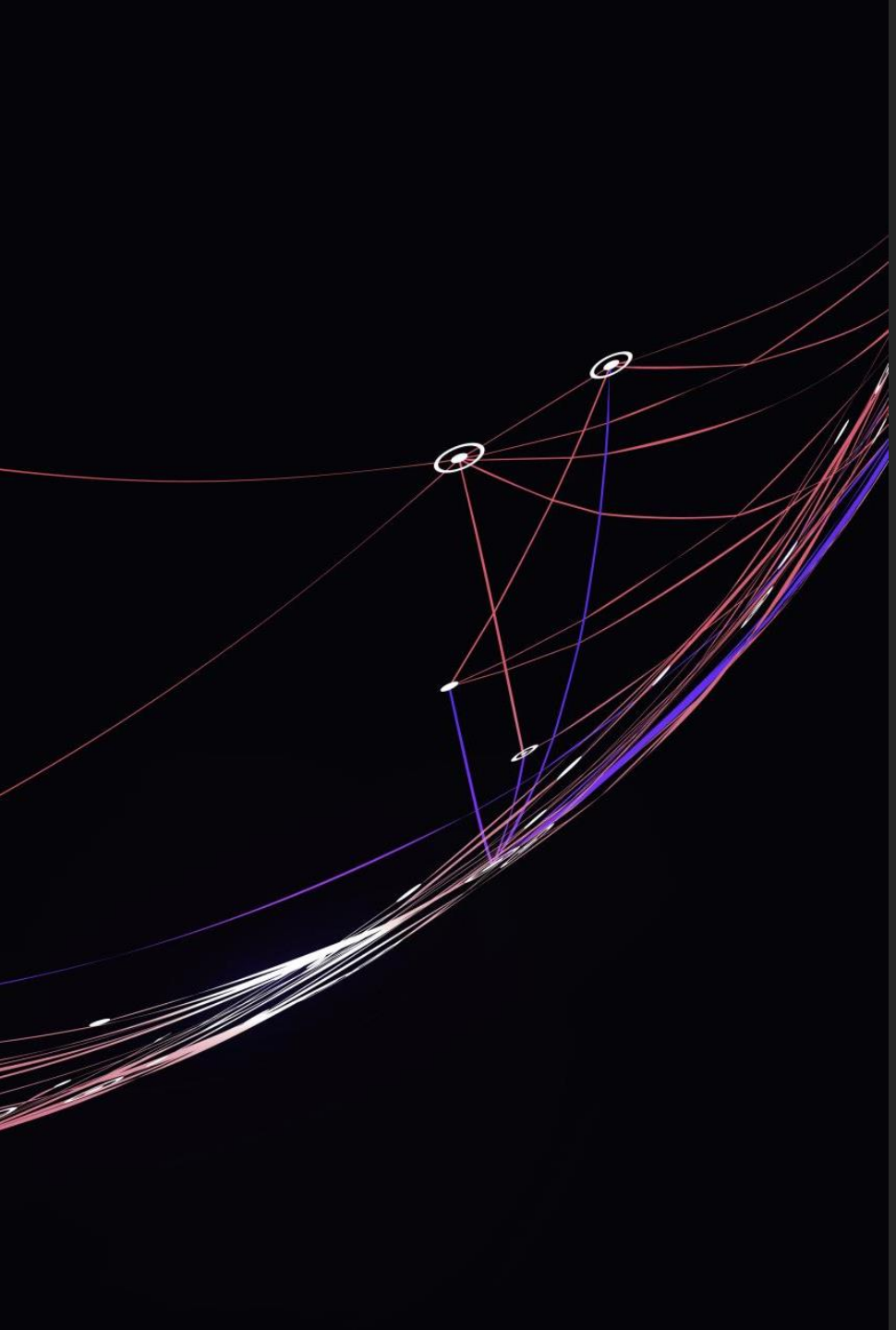
Integration of a Reinforcement Learning-Based Neural Network into the Crazyflie Flight Controller

ANDREA ARGNANI

MASSIMILIANO BILLI

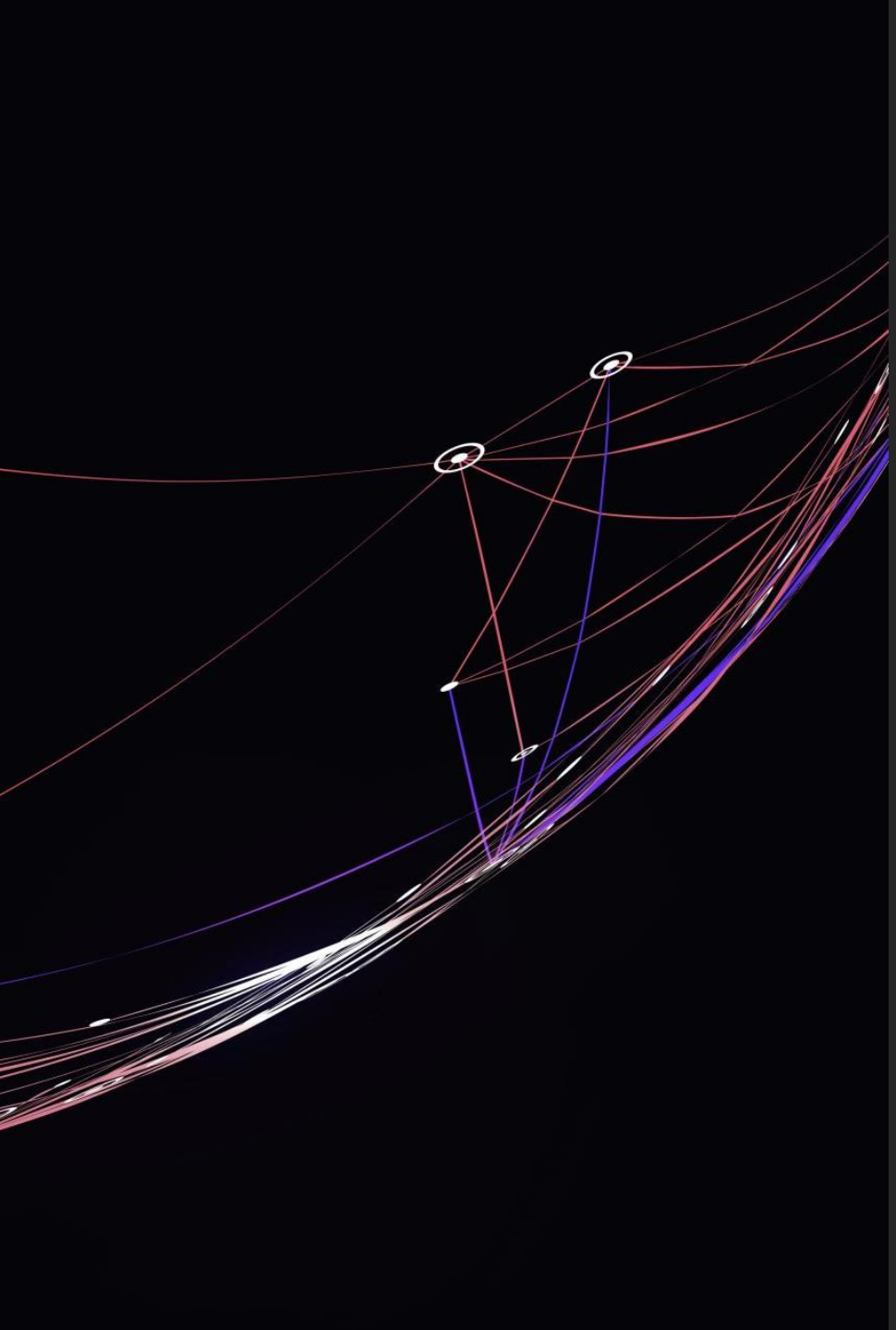
FEDERICO MAURI



Objective:

The goal of this project is to integrate an Artificial Neural Network (ANN), trained through Reinforcement Learning, into the Crazyflie flight platform to solve a hovering control task.

Interface the ANN with sensors and control stack. Evaluate execution time and memory loads.



Repository structure:

- *src/ANN_crazyflie.c*

  Contains the C implementation of the ANN.

- *get_header.py*

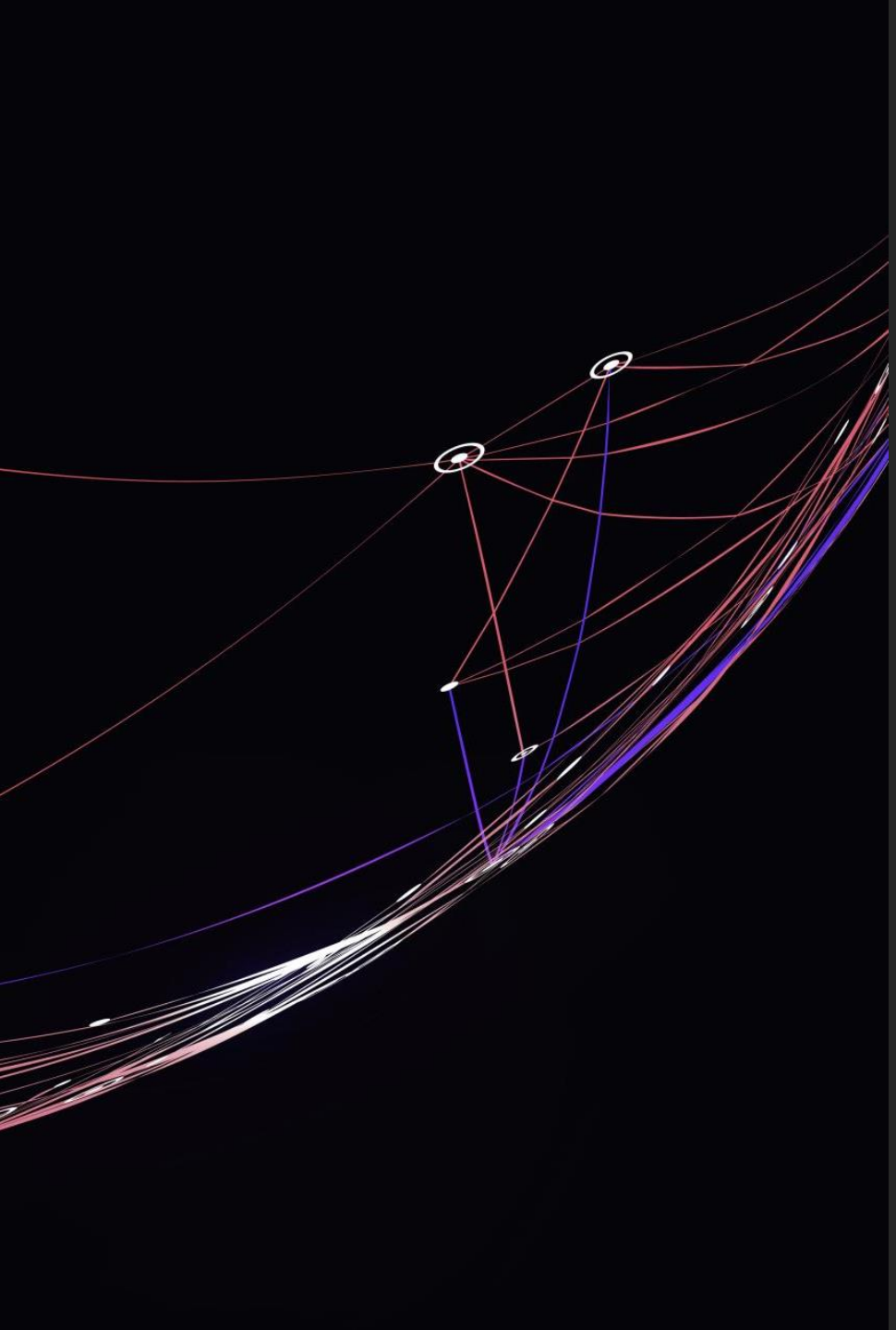
  Python script used to convert the model's weights, written in best_agent_noise01.pt in PyTorch format, into a C-compatible header file (src/pesi_modello.h).

- *golden_model.py*

  Python implementation of the same neural network used as a reference (golden model) to validate the C version's behavior of ANN_crazyflie.c.

- *get_header_random.py*

  Python script to generate random weights and layers to perform time analysis with different model sizes.



Implementation in appMain:

appMain structure:

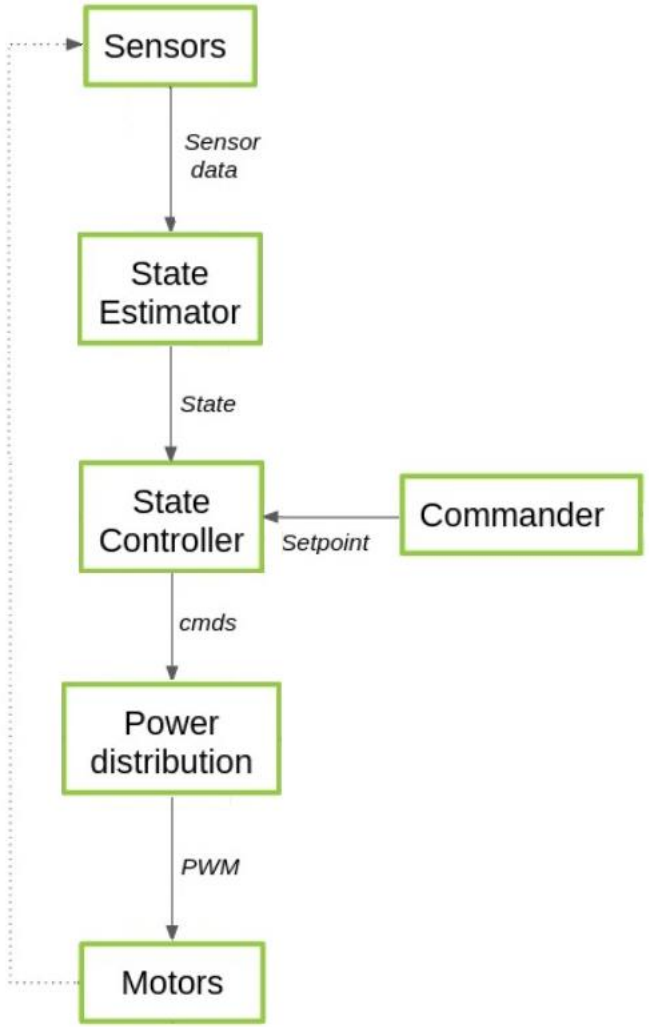
- Sensors reading
- Artificial Neural Network execution
- Motors control
  - Via Setpoint
  - Direct control via PWM

```
void setMotors ( float* outputsANN ){

  // Controllo diretto dei motori
  paramVarId_t idMotorPowerSetEnable = paramGetVarId("motorPowerSet", "enable");
  paramVarId_t idMotorPowerSetM1 = paramGetVarId("motorPowerSet", "m1");
  paramVarId_t idMotorPowerSetM2 = paramGetVarId("motorPowerSet", "m2");
  paramVarId_t idMotorPowerSetM3 = paramGetVarId("motorPowerSet", "m3");
  paramVarId_t idMotorPowerSetM4 = paramGetVarId("motorPowerSet", "m4");

  paramSetInt( idMotorPowerSetEnable, 1 );     // Nonzero to override controller w
  paramSetInt( idMotorPowerSetM1, (uint16_t) outputsANN[0] );
  paramSetInt( idMotorPowerSetM2, (uint16_t) outputsANN[1] );
  paramSetInt( idMotorPowerSetM3, (uint16_t) outputsANN[2] );
  paramSetInt( idMotorPowerSetM4, (uint16_t) outputsANN[3] );
}

void setSetPoint ( float* outputsANN ){

  setpoint_t setpoint;
  quaternion_t quaternion;

  quaternion.x = outputsANN[0];
  quaternion.y = outputsANN[1];
  quaternion.z = outputsANN[2];
  quaternion.w = outputsANN[3];

  setpoint.attitudeQuaternion = quaternion;
  setpoint.mode.quat = modeAbs;                // modeAbs o modeVelocity
  setpoint.velocity_body = true;

  commanderSetSetpoint(&setpoint, COMMANDER_PRIORITY_HIGHLEVEL );
}
```

Implementation in appMain:

appMain structure:

- Sensors reading
- Artificial Neural Network execution
- Motors control
  - Via Setpoint
  - Direct control via PWM



Implementation as Out-of-tree controller:

Used to run with higher execution priority

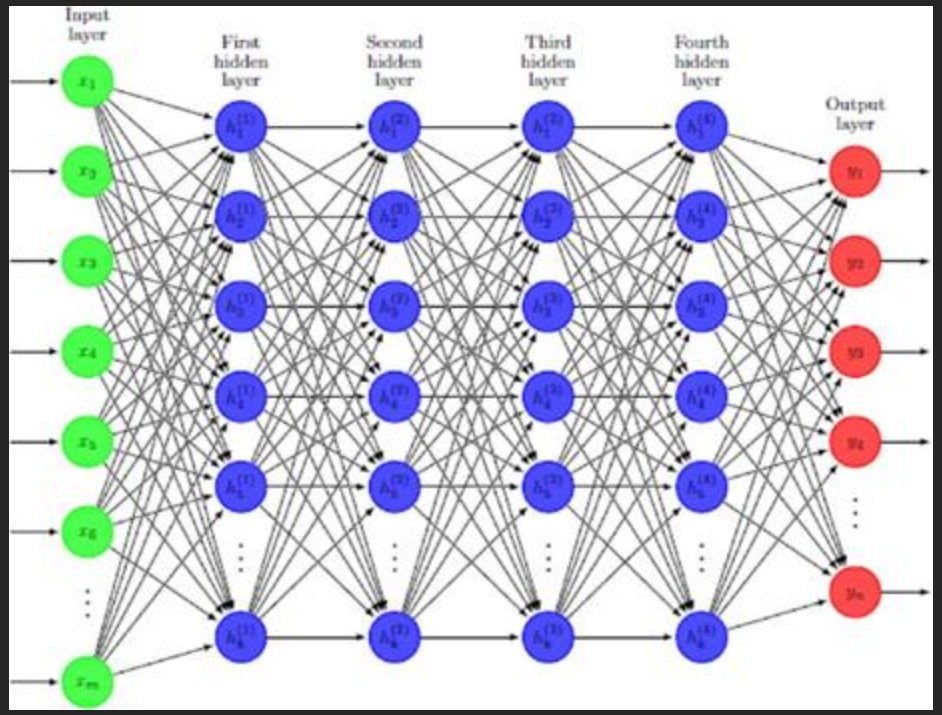
- Initialization with the *controllerOutOfTreeInit()*
- Tested using the golden model
- Realization of *controllerOutOfTree()*
  - Sensors reading
  - ANN execution
  - Motors control

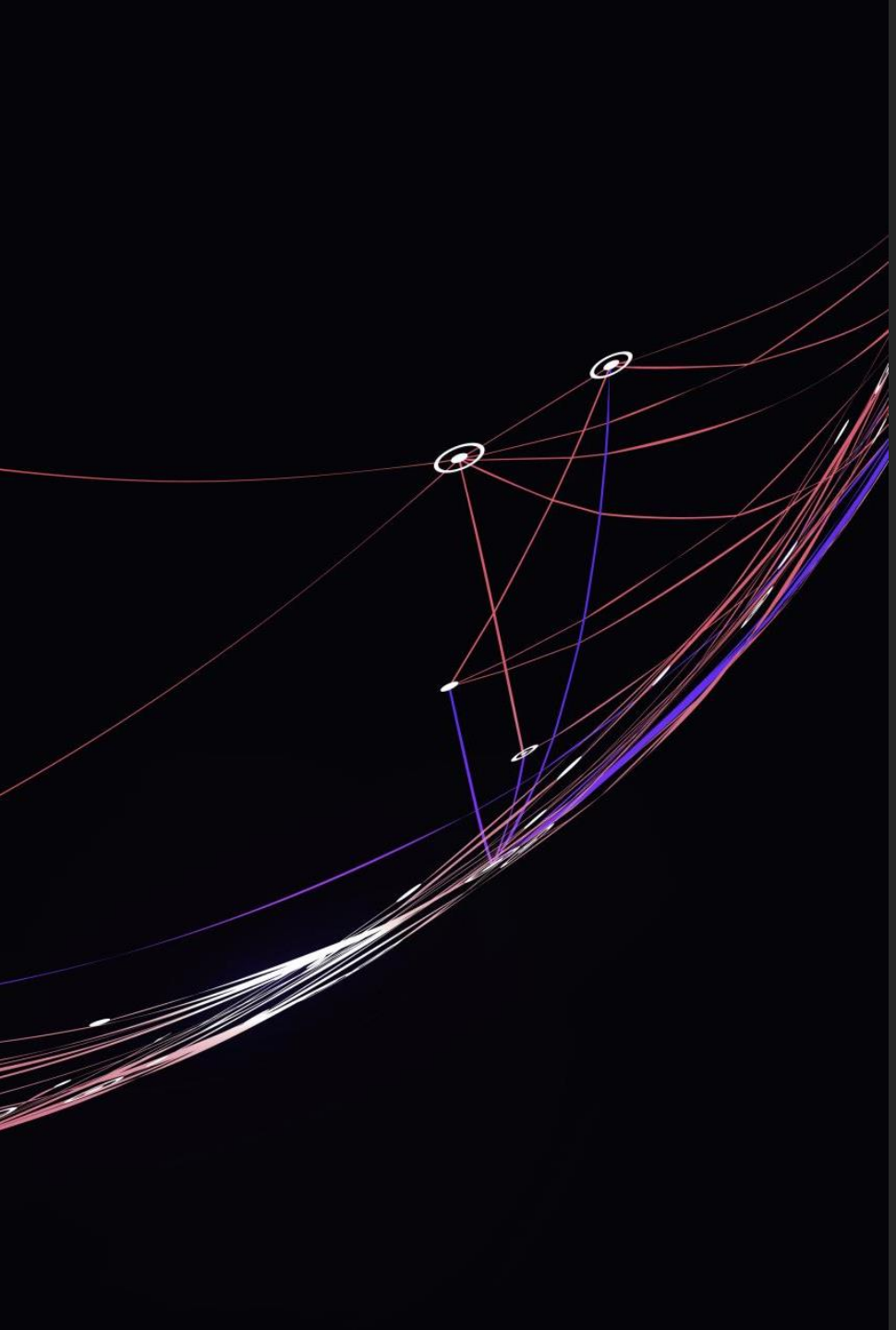
Artificial Neural Network

We used the ARM CMSIS-DSP library optimized for Cortex-M processors, to perform:

- Matrix-vector multiplications
- Vector additions

```
for( int l = 0 ; l < N_LAYER ; l++)
{
  int n = net_weights[l].numRows;
  arm_mat_vec_mult_f32( &net_weights[l], buffer_1, buffer_2);
  arm_add_f32( buffer_2, net_bias[l], buffer_1, n );
  // attivazione
  if( l < N_LAYER-1)
    relu_f32 ( buffer_1 , n );
}

for(int i=0; i<4; i++)
  output[i] = buffer_1[i];
```



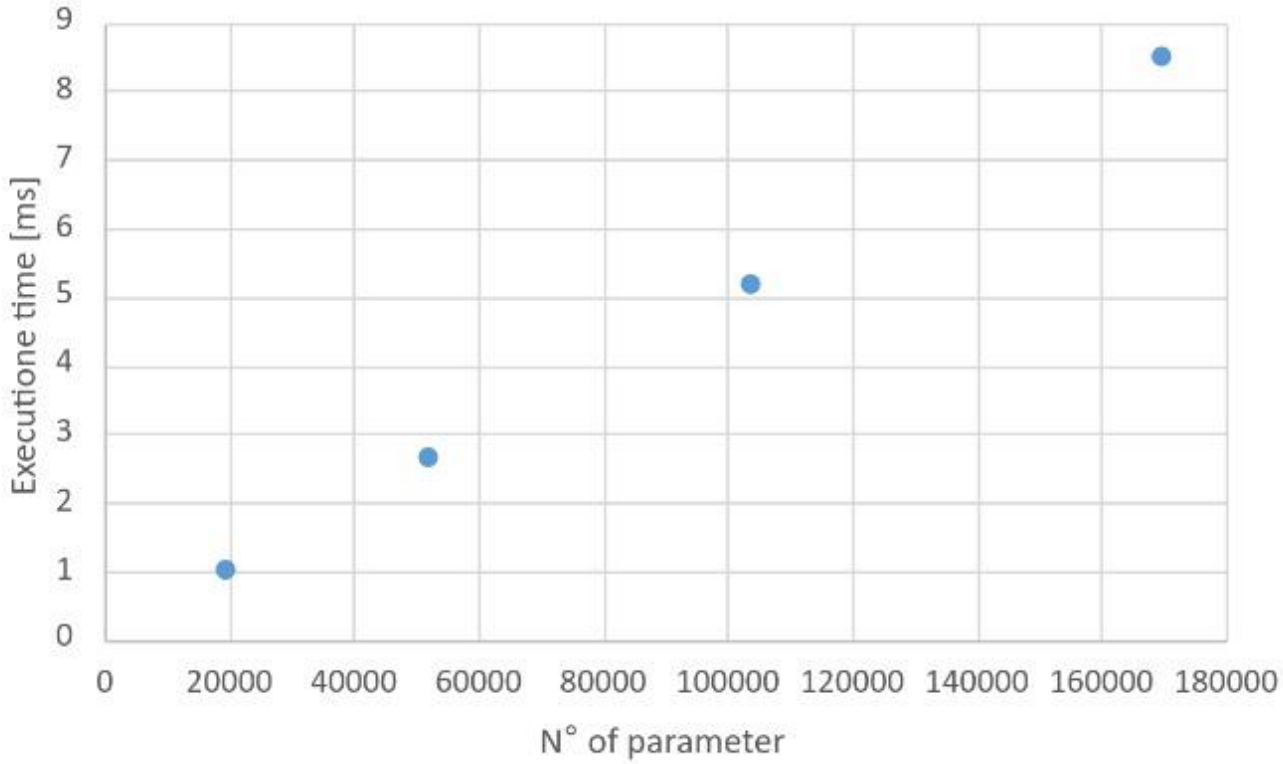


Memory occupation:

The artificial neural network (ANN) occupies approximately 400 KB, resulting in a total firmware size of 677 KB. Given that the Crazyflie's RAM is limited to only 196 KB, we decided to store the ANN weights in the FLASH memory (1 MB). To achieve this, we declared the weights using the *const* keyword.

Time analysis:

We generated multiple models with varying numbers of layers and total parameters using the *get_header_random.py* script, in order to evaluate the trade-off between the ANN's complexity and its execution time.



Layer 1	Layer 2	Layer 3	Layer 4	N° pesi	Dim. Modello	Dim. Totale	Tempo [ms]
128	128			19332	76 kB	347 kB	1,01
128	256	64		52036	203 kB	474 kB	2,65
256	256	128		103812	406 kB	677 kB	5,2
256	256	256	128	169604	663 kB	934 kB	8,52

Future developments:

- Ensure that the Crazyradio link operates correctly when an out-of-tree controller is in use.
- Optimize the execution of the ANN via DMA
- Implementation on the AI-Deck

Thanks for the attention